UNIVERSITATEA NAȚIONALĂ de ȘTIINȚĂ și TEHNOLOGIE POLITEHNICA din BUCUREȘTI

Facultatea de Electronică, Telecomunicaţii şi Tehnologia Informaţiei

Baze de **D**ate **- P**rogramarea **I**nterfețelor

~ **Carte de Bucate** - Proiect_JSB (Java Spring Boot) ~

Student: Necula Adrian-Nicușor

Coordonator: Ș.l./Lect. Dr. Ing. Valentin Pupezescu

Grupa 433E

București 2025 – 2026

Cuprins

1. Scopul și Obiectivele Proiectului……..

2. Tehnologii și Instrumente Utilizate……...

2.1. Sistemul de Gestiune a Bazelor de Date: MySQL……

2.2. Limbajul de programare Java și Ecosistemul Spring……

2.3. Motorul de template-uri Thymeleaf……..

2.4. Mediul de dezvoltare și unelte (Maven, IDE) ……..

3. Arhitectura Aplicației și Structura Datelor……..……..

3.1. Arhitectura MVC (Model-View-Controller) ……..……..

3.2. Structura Bazei de Date (Schema EER) ……..…….

3.3. Descrierea Entităților……..…….....

4. Descrierea Funcționalităților și a Fluxului de Lucru…...…….....

5. Implementarea Funcțiilor……..

5.1. Gestionare Rețete (Entitatea Principală)……...

5.2. Gestionare Ingrediente (Entitatea Secundară)……..

5.3. Gestionare Compoziție Rețete (Relația Many-to-Many)…….....

6. Interfața Grafică și Design (Front-End)……..…….....

7. Concluzii……...

8. Bibliografie……

1. Scopul și Obiectivele Proiectului

În contextul actual al digitalizării informațiilor, scopul acestui proiect este proiectarea și implementarea unei aplicații web robuste pentru gestionarea unei "Cărți de Bucate Digitale". Aplicația își propune să înlocuiască metodele tradiționale de stocare a rețetelor (formate fizice sau fișiere text neorganizate) cu o soluție centralizată, persistentă și ușor de interogat, cu ajutorul tehnologiei **Java Spring Boot.**

Obiectivul principal este realizarea unei interfețe prietenoase care să permită utilizatorului (bucătar sau administrator) să gestioneze un catalog complex de preparate culinare. Proiectul evidențiază capacitatea de a construi un sistem software complet, pornind de la proiectarea bazei de date relaționale și ajungând la implementarea logicii de business și a interfeței cu utilizatorul.

Specificul și complexitatea acestui proiect rezidă în gestionarea relațiilor dintre date. O rețetă nu este o entitate izolată, ci un ansamblu de ingrediente, fiecare având caracteristici proprii (nume, alergeni). Mai mult, legătura dintre rețetă și ingredient nu este una simplă, ci necesită atribute contextuale: cantitatea și unitatea de măsură (ex: "200 de grame" de făină pentru o prăjitură, dar "1 lingură" de făină pentru un sos). Astfel, aplicația demonstrează manipularea corectă a relațiilor de tip Many-to-Many (M:N) cu atribute suplimentare pe tabelul de asociere.

2. Tehnologii și Instrumente Utilizate

Pentru dezvoltarea aplicației au fost selectate tehnologii moderne, standardizate în industria software, care asigură scalabilitate, securitate și ușurință în mentenanță.

2.1. Sistemul de Gestiune a Bazelor de Date: MySQL

Persistența datelor este asigurată de MySQL, cel mai popular sistem de gestiune a bazelor de date relaționale (RDBMS) open-source. MySQL oferă ca avantaje:

- Integritate Referențială: Suportul robust pentru chei străine (Foreign Keys) și constrângeri (Constraints), esențiale pentru a preveni ștergerea accidentală a ingredientelor folosite în rețete active.
- Performanță: Capacitatea de a executa rapid interogări complexe, inclusiv JOIN-uri între tabelele de rețete, ingrediente și asocieri.
- Compatibilitate: Integrarea nativă excelentă cu driverele JDBC din Java.

2.2. Limbajul de programare Java și Ecosistemul Spring

Logica de backend a fost implementată utilizând Java împreună cu framework-ul Spring Boot.

- Spring Boot: A fost ales pentru capacitatea sa de a simplifica configurarea aplicațiilor Spring ("convention over configuration"). Acesta elimină necesitatea fișierelor XML complexe de

configurare, oferind un server web integrat (Apache Tomcat) care permite rularea aplicației ca un fișier .jar independent.

- Spring Data JPA: Această componentă abstractizează interacțiunea cu baza de date. Prin utilizarea interfețelor Repository (extinzând JpaRepository), operațiile SQL standard (SELECT, INSERT, UPDATE, DELETE) sunt generate automat, reducând considerabil cantitatea de cod ("boilerplate code") necesară.
- Hibernate: Este utilizat ca implementare a standardului JPA (Java Persistence API), realizând maparea obiect-relațional (ORM). Acesta transformă automat înregistrările din tabelele MySQL în obiecte Java și invers.

2.3. Motorul de template-uri Thymeleaf

Pentru partea de prezentare (View), am utilizat Thymeleaf. Acesta este un motor de template-uri Java modern, care permite generarea dinamică a paginilor HTML pe server. Spre deosebire de JSP, Thymeleaf permite păstrarea structurii naturale a HTML-ului, fișierele putând fi vizualizate corect chiar și în absența serverului (ca prototipuri statice). Integrarea profundă cu Spring permite legarea ușoară a obiectelor din backend (Model) de elementele din interfață (input-uri, tabele).

2.4. Mediul de dezvoltare și unelte (Maven, IDE)

- Maven: Utilizat pentru gestionarea dependențelor proiectului (bibliotecile Spring, driverul MySQL, Thymeleaf) și pentru procesul de compilare.
- IDE (Eclipse/IntelliJ): Mediul integrat de dezvoltare folosit pentru scrierea codului, depanare și rularea serverului local.

3. Arhitectura Aplicației și Structura Datelor

3.1. Arhitectura MVC (Model-View-Controller)

Aplicația respectă cu strictețe șablonul arhitectural MVC, care asigură o separare clară a responsabilităților:

- Model (Modelul de date): Reprezentat de clasele Entity (Reteta, Ingredient, RetetaIngredient). Acestea sunt mapate direct peste tabelele din baza de date și conțin doar date și metode de acces (Getters/Setters).
- View (Interfața): Fișierele HTML/Thymeleaf situate în directorul templates. Acestea sunt responsabile doar cu afișarea datelor primite de la Controller.
- Controller (Logica de control): Clasele Java adnotate cu @Controller. Acestea interceptează cererile utilizatorului (HTTP GET/POST), apelează serviciile necesare pentru procesarea datelor și decid ce pagină (View) să returneze utilizatorului.

Suplimentar, arhitectura include un strat de Service (@Service), interpus între Controller și Repository, unde este implementată logica de business complexă, asigurând o decuplare și mai bună a componentelor.

3.2. Structura Bazei de Date (Schema EER)

Baza de date, denumită generic proiect, este normalizată pentru a elimina redundanța datelor. Diagrama EER (Enhanced Entity-Relationship) evidențiază cele trei tabele și relațiile dintre ele.

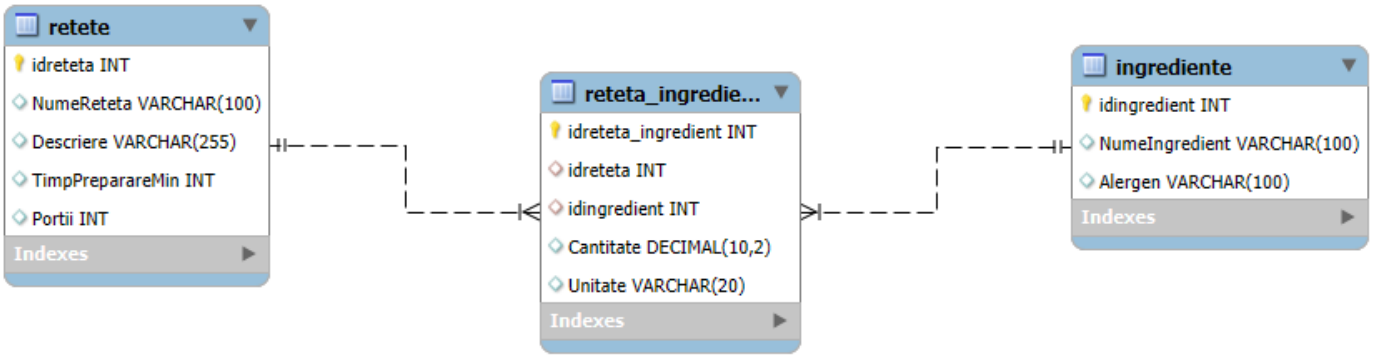

Fig. 1 – Diagrama EER a bazei de date, evidențiind relația M:N descompusă

3.3. Descrierea Entităților

- Tabela retete: Reprezintă entitatea centrală.
  - idreteta (INT, PK, Auto Increment): Identificator unic.
  - NumeReteta (VARCHAR): Denumirea preparatului.
  - Descriere (VARCHAR): Instrucțiuni de preparare.
  - TimpPrepareMin (INT): Durata în minute.
  - Portii (INT): Numărul de porții rezultate.
- Tabela ingrediente: Reprezintă nomenclatorul de materii prime.
  - idingredient (INT, PK, Auto Increment): Identificator unic.
  - NumeIngredient (VARCHAR): Ex: "Făină", "Ouă".
  - Alergen (VARCHAR): Informații despre potențiali alergeni (Ex: "Gluten").
- Tabela reteta_ingredient: Tabela de asociere (Junction Table).
  - idreteta_ingredient (INT, PK): ID propriu al asocierii.
  - idreteta (INT, FK): Referință către rețetă.
  - idingredient (INT, FK): Referință către ingredient.
  - Cantitate (DECIMAL/VARCHAR): Cât se folosește din ingredientul respectiv.
  - Unitate (VARCHAR): Unitatea de măsură (grame, bucăți, ml).

4. Descrierea Funcționalităților și a Fluxului de Lucru

Aplicația este structurată sub forma unui Dashboard centralizat. La accesarea paginii principale, utilizatorul este întâmpinat de un meniu intuitiv împărțit în trei secțiuni majore, corespunzătoare celor trei tabele din baza de date.

Fluxul de lucru general pentru utilizator este următorul:

- Definirea Ingredientelor: Utilizatorul populează mai întâi baza de date cu ingredientele disponibile (ex: sare, piper, carne).
- Definirea Rețetelor: Utilizatorul creează "antetul" rețetei (nume, timp, descriere), fără a specifica încă ingredientele.
- Realizarea Asocierilor (Compoziția): În secțiunea "Compoziție Rețete", utilizatorul leagă ingredientele de rețete, specificând cantitățile.

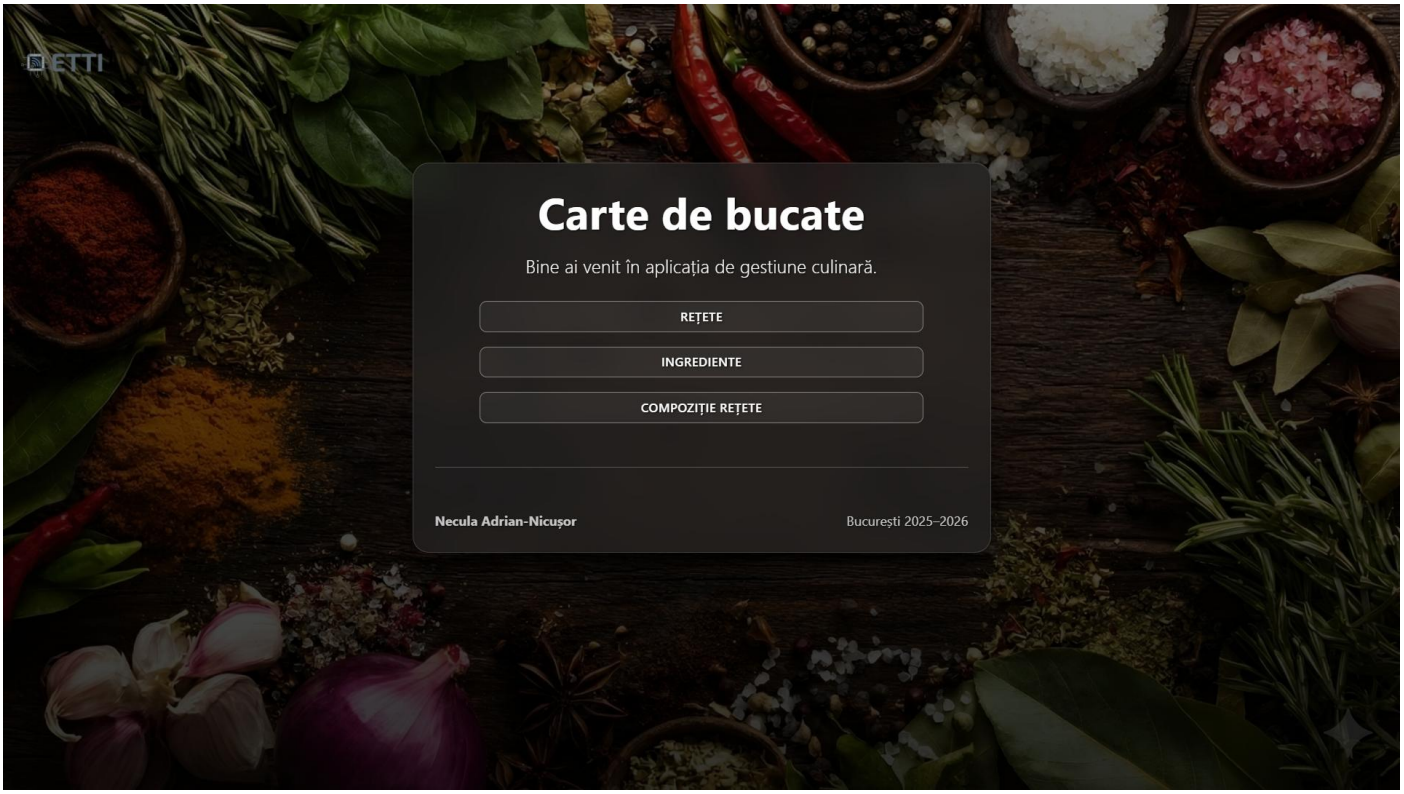

Fig. 2 – Pagina principală (Dashboard) a aplicației web

5. Implementarea funcțiilor

În această secțiune sunt prezentate funcționalitățile principale ale aplicației, împreună cu fragmente de cod reprezentative care evidențiază modul de implementare. Conform cerinței, nu este inclus întregul cod al proiectului, ci doar porțiunile relevante pentru operațiile de bază asupra datelor (vizualizare, adăugare, modificare şi ştergere), atât pentru tabelele principale, cât și pentru tabela de legătură.

Fragmentele selectate acoperă rutele din controller, interacțiunea cu baza de date prin stratul de service/repository şi modul în care datele sunt transmise către paginile HTML (Thymeleaf) pentru afişare și formulare.

Interfața pentru fiecare operație de tip CRUD este similară pentru fiecare tabelă în parte, precum se poate vedea în următoarele figuri:

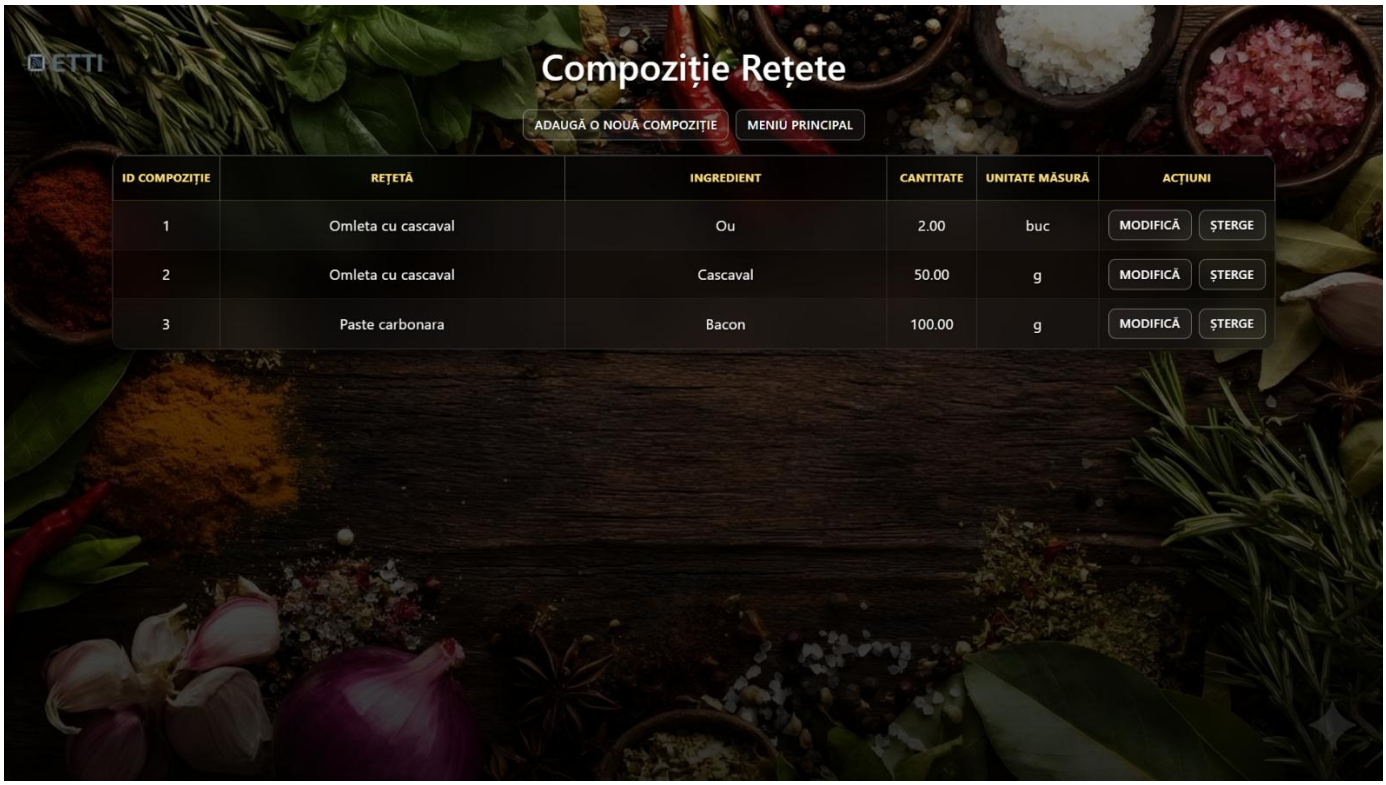

Fig. 3 – Meniul tabelei asociative (M:N) Compoziție Rețete

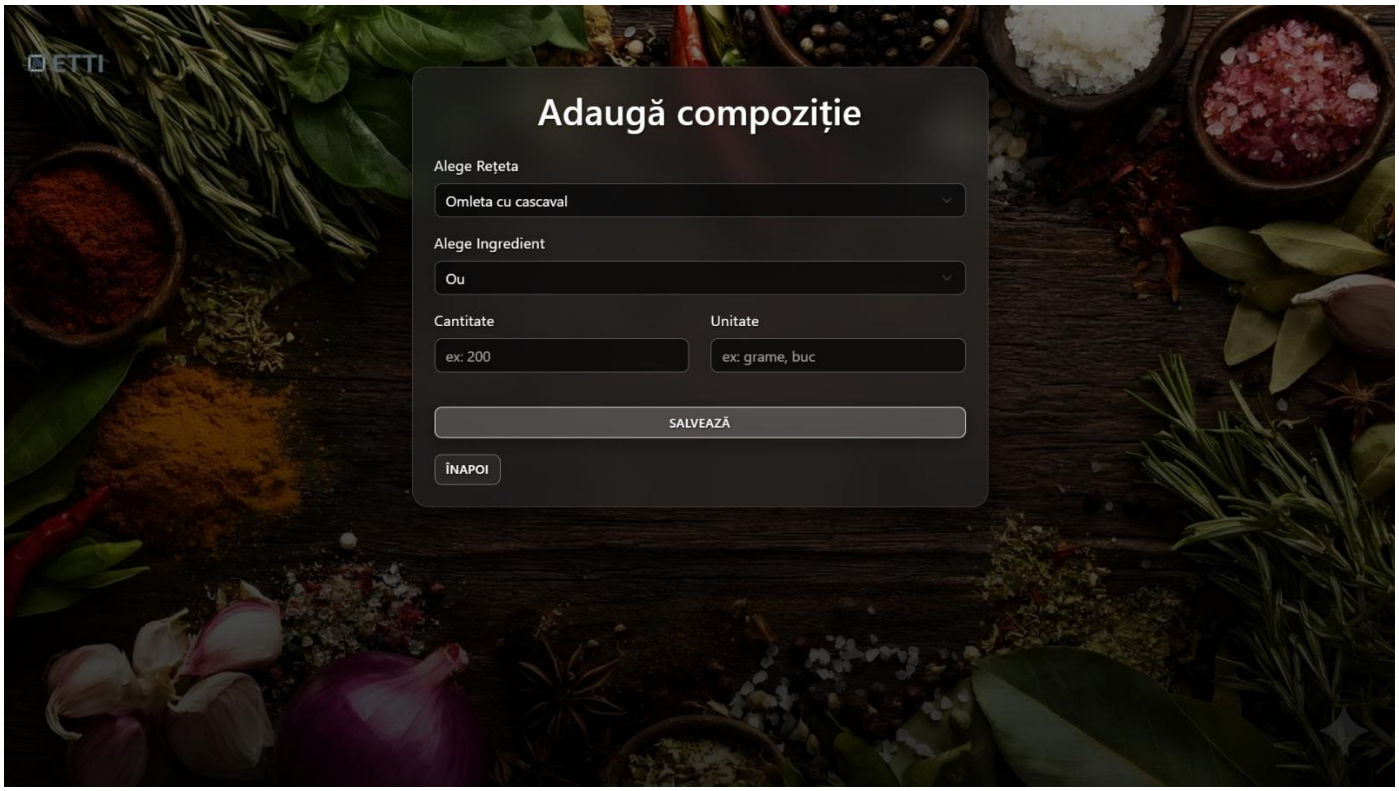

Fig. 4 – Meniul pentru funcția de adăugare a unei noi asocieri

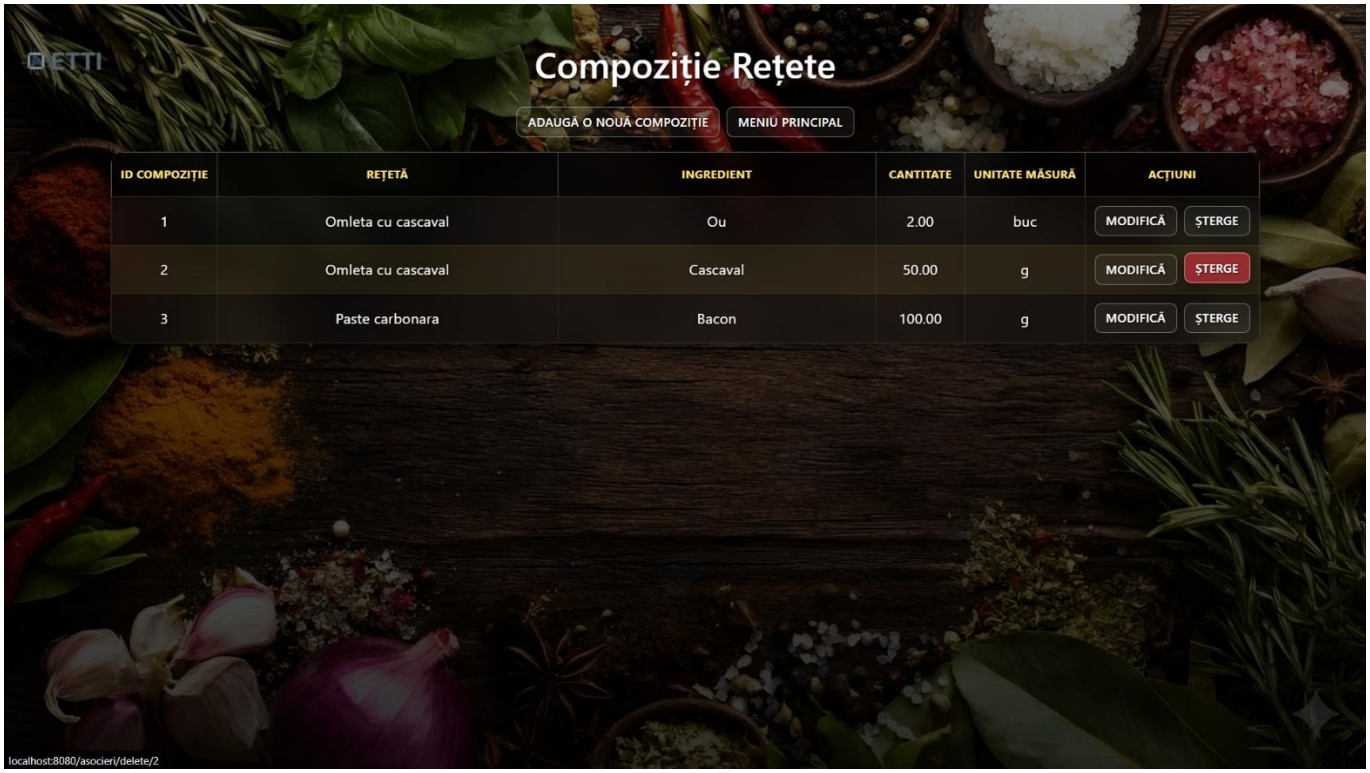

Fig. 5 – Meniul pentru selectarea și efectuarea funcției de ștergere

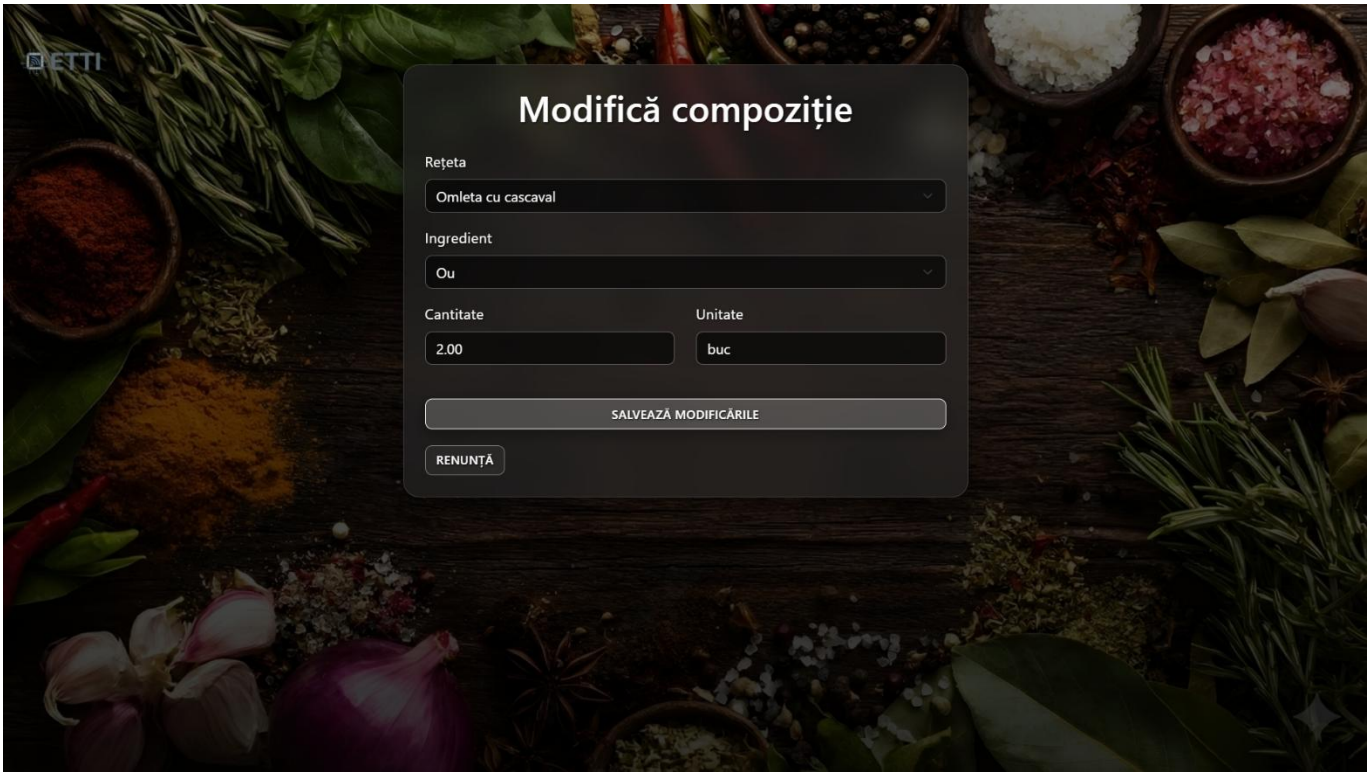

Fig. 6 – Meniul pentru funcția de modificare a asocierii selectate

5.1. Gestionare Rețete (Entitatea Principală)

Modulul **Rețete** permite gestionarea completă a înregistrărilor din **tabela retete**. Funcționalitățile implementate sunt: listare, adăugare, editare și ștergere rețete.

Interacțiunea se face prin pagini Thymeleaf (**retete.html**, **create_reteta.html**, **edit_reteta.html**) și rute gestionate de **RetetaController**.

Pentru adăugarea unei rețete, utilizatorul completează formularul din **create_reteta.html** și apasă butonul de salvare. La trimiterea formularului, datele sunt trimise către ruta **/retete** (metoda POST) şi sunt preluate în controller prin *@ModelAttribute*. Controller-ul apelează **retetaService.save(reteta)** pentru salvarea în baza de date.

```
<form th:action="@{/retete}" th:object="${reteta}" method="post">

  <div class="mb-3">
    <label class="form-label">Nume rețetă</label>
    <input type="text"
           class="form-control"
           placeholder="Introdu numele rețetei"
           th:field="*{numeReteta}"
           required />
  </div>

  <div class="mb-3">
    <label class="form-label">Descriere</label>
    <textarea class="form-control"
              rows="3"
              placeholder="Descriere scurtă (opțional)"
              th:field="*{descriere}"></textarea>
  </div>

  <div class="row">
    <div class="col-md-6 mb-3">
      <label class="form-label">Timp preparare (minute)</label>
      <input type="number"
             class="form-control"
             min="0"
             placeholder="Ex: 25"
             th:field="*{timpPrepareMin}" />
    </div>

    <div class="col-md-6 mb-3">
      <label class="form-label">Porții</label>
      <input type="number"
             class="form-control"
             min="0"
             placeholder="Ex: 2"
             th:field="*{portii}" />
    </div>
  </div>

  <div class="d-grid gap-2 mt-4">
    <button type="submit" class="btn btn-primary btn-lg">
      Salvează
    </button>
  </div>

  <div class="d-flex justify-content-between mt-3">
    <a class="btn btn-outline-secondary" th:href="@{/retete}">
      Înapoi la listă
    </a>
    <a class="btn btn-outline-dark" th:href="@{/}">
      Meniu principal
    </a>
  </div>

</form>
```

*Fig. 6 – Formularul de adăugare rețetă (**create_reteta.html**): utilizatorul completează câmpurile, iar la apăsarea butonului „Salvează” formularul trimite datele către ruta **/retete** prin metoda POST*

```
@GetMapping("/retete/new")
public String createRetetaForm(Model model) {
    model.addAttribute("reteta", new Reteta());
    return "create_reteta";
}

@PostMapping("/retete")
public String saveReteta(@ModelAttribute("reteta") Reteta reteta) {
    retetaService.save(reteta);
    return "redirect:/retete";
}
```

*Fig. 7 – Metoda din **RetetaController** asociată rutei POST **/retete**, care preia datele din formular și salvează rețeta în baza de date*

Pentru editarea unei rețete, aplicația deschide formularul de editare prin ruta GET **/retete/edit/{id}**, care încarcă datele rețetei selectate. La salvare, formularul trimite datele către ruta POST **/retete/{id}**, iar controller-ul actualizează câmpurile și salvează modificările în baza de date.

```
<form th:action="@{/retete/{id}(id=${reteta.idReteta})}" th:object="${reteta}" method="post">

  <div class="mb-3">
      <label class="form-label">ID Rețetă</label>
      <input type="text" class="form-control" th:value="*{idReteta}" disabled />
  </div>

  <div class="mb-3">
    <label class="form-label">Nume rețetă</label>
    <input type="text" class="form-control" th:field="*{numeReteta}" required />
  </div>

  <div class="mb-3">
    <label class="form-label">Descriere</label>
    <textarea class="form-control" rows="3" th:field="*{descriere}"></textarea>
  </div>

  <div class="row">
    <div class="col-md-6 mb-3">
      <label class="form-label">Timp preparare (minute)</label>
      <input type="number" class="form-control" min="0" th:field="*{timpPrepareMin}" />
    </div>
    <div class="col-md-6 mb-3">
      <label class="form-label">Porții</label>
      <input type="number" class="form-control" min="0" th:field="*{portii}" />
    </div>
  </div>

  <div class="d-grid gap-2 mt-4">
    <button type="submit" class="btn btn-primary btn-lg">Salvează modificările</button>
  </div>

  <div class="d-flex justify-content-between mt-3">
    <a class="btn btn-outline-secondary" th:href="@{/retete}">Renunță</a>
  </div>

</form>
```

*Fig. 8 – Formularul de editare rețetă (**edit_reteta.html**): la apăsarea butonului „Salvează Modificările”, datele sunt trimise către ruta **/retete/{id}** prin metoda POST*

```
@GetMapping("/retete/edit/{idreteta}")
public String editRetetaForm(@PathVariable Integer idreteta, Model model) {
    Reteta reteta = retetaService.findById(idreteta).orElse(null);
    if (reteta == null) {
        return "redirect:/retete";
    }
    model.addAttribute("reteta", reteta);
    return "edit_reteta";
}

@PostMapping("/retete/{idreteta}")
public String updateReteta(@PathVariable Integer idreteta,
                           @ModelAttribute("reteta") Reteta reteta) {
    Reteta existenta = retetaService.findById(idreteta).orElse(null);
    if (existenta == null) {
        return "redirect:/retete";
    }

    existenta.setNumeReteta(reteta.getNumeReteta());
    existenta.setDescriere(reteta.getDescriere());
    existenta.setTimpPrepareMin(reteta.getTimpPrepareMin());
    existenta.setPortii(reteta.getPortii());

    retetaService.save(existenta);
    return "redirect:/retete";
}
```

*Fig. 9 – Implementarea editării în **RetetaController**: ruta GET **/retete/edit/{id}** încarcă datele rețetei în formular, iar ruta POST **/retete/{id}** preia modificările și actualizează înregistrarea*

Pentru ștergerea unei rețete, utilizatorul apasă butonul „Șterge” din tabela de rețete. Înainte de executare, interfața afișează o confirmare pentru a evita ștergerile accidentale. Dacă utilizatorul confirmă, se apelează ruta **/retete/delete/{id}**, iar controller-ul șterge înregistrarea și revine la lista de rețete.

```
<a class="btn btn-outline-danger btn-sm"
   th:href="@{/retete/delete/{id}(id=${reteta.idReteta})}"
   onclick="return confirm('Sigur vrei să ștergi această rețetă?');">
  Șterge
</a>
```

*Fig. 10 – Butonul „Șterge” din lista de rețete: la confirmare, este apelată ruta **/retete/delete/{id}** pentru rețeta* selectată

```
@GetMapping("/retete/delete/{idreteta}")
public String deleteReteta(@PathVariable Integer idreteta) {
    retetaService.deleteById(idreteta);
    return "redirect:/retete";
}
```

*Fig. 11 – Implementarea ștergerii în **RetetaController**: ruta GET **/retete/delete/{id}** apelează serviciul de ștergere și redirecționează către lista de rețete*

5.2. Gestionare Ingrediente (Entitatea Secundară)

Modulul **Ingrediente** este responsabil pentru administrarea nomenclatorului de materii prime utilizate în preparate. Acesta asigură consistența datelor, permițând definirea ingredientelor (nume, alergeni) o singură dată și reutilizarea lor în multiple rețete.

Interfața este gestionată prin pagini Thymeleaf (**ingrediente.html**, **create_ingredient.html**, **edit_ingredient.html**), iar logica de business este implementată în **IngredientController**.

Pentru adăugarea unui ingredient, utilizatorul completează formularul din **create_ingredient.html** și apasă butonul de salvare. La trimiterea formularului, datele sunt trimise către ruta **/ingrediente** (metoda POST) și sunt preluate în controller prin *@ModelAttribute*. Controller-ul preia obiectul populat și apelează **ingredientService.save(ingredient)** pentru salvarea în baza de date.

```
<form th:action="@{/ingrediente}" th:object="${ingredient}" method="post">

  <div class="mb-3">
    <label class="form-label">Nume ingredient</label>
    <input type="text"
           class="form-control"
           placeholder="Introdu numele ingredientului"
           th:field="*{numeIngredient}"
           required />
  </div>

  <div class="mb-3">
    <label class="form-label">Alergen</label>
    <input type="text"
           class="form-control"
           placeholder="Ex: ouă, lapte, gluten (sau lasă gol)"
           th:field="*{alergen}" />
  </div>

  <div class="d-grid gap-2 mt-4">
    <button type="submit" class="btn btn-primary btn-lg">
      Salvează
    </button>
  </div>

  <div class="d-flex justify-content-between mt-3">
    <a class="btn btn-outline-secondary" th:href="@{/ingrediente}">
      Înapoi la listă
    </a>
    <a class="btn btn-outline-dark" th:href="@{/}">
      Meniu principal
    </a>
  </div>

</form>
```

*Fig. 12 – Formularul de adăugare ingredient (**create_ingredient.html**): utilizatorul completează câmpurile, iar la apăsarea butonului „Salvează” formularul trimite datele către ruta **/ingrediente** prin metoda POST*

```java
@GetMapping("/ingrediente/new")
public String createIngredientForm(Model model) {
    model.addAttribute("ingredient", new Ingredient());
    return "create_ingredient";
}

@PostMapping("/ingrediente")
public String saveIngredient(@ModelAttribute("ingredient") Ingredient ingredient) {
    ingredientService.save(ingredient);
    return "redirect:/ingrediente";
}
```

*Fig. 13 – Metoda din **IngredientController** asociată rutei POST **/ingrediente**, care preia datele din formular și salvează ingredientul în baza de date*

Pentru editarea unui ingredient, aplicația deschide formularul de editare prin ruta GET **/ingrediente/edit/{id}**, care încarcă datele ingredientului selectat folosind serviciul **findById**. La salvare, formularul trimite datele către ruta POST **/ingrediente/{id}**, iar controller-ul actualizează câmpurile obiectului existent (Nume Ingredient, Alergen) și salvează modificările în baza de date.

```html
<form th:action="@{/ingrediente/{id}(id=${ingredient.idIngredient})}"
      th:object="${ingredient}"
      method="post">

  <div class="mb-3">
      <label class="form-label">ID Ingredient</label>
      <input type="text" class="form-control" th:value="*{idIngredient}" disabled />
  </div>

  <div class="mb-3">
    <label class="form-label">Nume ingredient</label>
    <input type="text" class="form-control" th:field="*{numeIngredient}" required />
  </div>

  <div class="mb-3">
    <label class="form-label">Alergen</label>
    <input type="text" class="form-control" th:field="*{alergen}" />
  </div>

  <div class="d-grid gap-2 mt-4">
    <button type="submit" class="btn btn-primary btn-lg">Salvează modificările</button>
  </div>

  <div class="d-flex justify-content-between mt-3">
    <a class="btn btn-outline-secondary" th:href="@{/ingrediente}">Renunță</a>
  </div>

</form>
```

*Fig. 14 – Formularul de editare ingredient (**edit_ingredient.html**): la apăsarea butonului „Salvează modificările”, datele sunt trimise către ruta **/ingrediente/{id}** prin metoda POST*

```java
@GetMapping("/ingrediente/edit/{idingredient}")
public String editIngredientForm(@PathVariable Integer idingredient, Model model) {
    Ingredient ingredient = ingredientService.findById(idingredient).orElse(null);
    if (ingredient == null) return "redirect:/ingrediente";

    model.addAttribute("ingredient", ingredient);
    return "edit_ingredient";
}

@PostMapping("/ingrediente/{idingredient}")
public String updateIngredient(@PathVariable Integer idingredient,
                               @ModelAttribute("ingredient") Ingredient ingredient) {

    Ingredient existent = ingredientService.findById(idingredient).orElse(null);
    if (existent == null) return "redirect:/ingrediente";

    existent.setNumeIngredient(ingredient.getNumeIngredient());
    existent.setAlergen(ingredient.getAlergen());

    ingredientService.save(existent);
    return "redirect:/ingrediente";
}
```

*Fig. 15 – Implementarea editării în **IngredientController**: ruta GET încarcă datele, iar ruta POST preia modificările și actualizează entitatea*

Pentru ștergerea unui ingredient, utilizatorul apasă butonul „Șterge” din tabela de ingrediente. Înainte de executare, interfața afișează o confirmare JavaScript (**return confirm(...)**) pentru a evita ștergerile accidentale. Dacă utilizatorul confirmă, se apelează ruta **/ingrediente/delete/{id}**, iar controller-ul șterge înregistrarea și revine la lista de ingrediente.

```
<a class="btn btn-outline-danger btn-sm"
   th:href="@{/ingrediente/delete/{id}(id=${ingredient.idIngredient})}"
   onclick="return confirm('Sigur vrei să ștergi acest ingredient?');">
  Șterge
</a>
```

Fig. 16 – Butonul „Șterge” din lista de ingrediente: la confirmare, este apelată ruta ***/ingrediente/delete/{id}***

```
@GetMapping("/ingrediente/delete/{idingredient}")
public String deleteIngredient(@PathVariable Integer idingredient) {
    ingredientService.deleteById(idingredient);
    return "redirect:/ingrediente";
}
```

Fig. 17 – Implementarea ștergerii în ***IngredientController****: metoda apelată prin GET primește ID-ul și execută ștergerea prin service*

5.3. Gestionare Compoziție Rețete (Relația Many-to-Many)

Modulul Compoziție Rețete gestionează asocierea dintre rețete și ingrediente, utilizând tabela de legătură **reteta_ingredient** (relație de tip M:N). Funcționalitățile implementate sunt: listare, adăugare, editare și ștergere înscrieri, împreună cu afișarea datelor referite (rețetă + ingredient).

Interacțiunea se face prin pagini Thymeleaf (**asocieri.html**, **create_asociere.html**, **edit_asociere.html**) și rute gestionate de **RetetaIngredientController**.

Pentru gestionarea înscrierilor, aplicația permite asocierea unei rețete cu un ingredient, deoarece relația dintre cele două tabele este de tip many-to-many (M:N) și este modelată prin tabela intermediară **reteta_ingredient**. Utilizatorul poate crea o înscriere nouă selectând rețeta și ingredientul din liste, stabilind cantitatea și unitatea de măsură, iar aplicația salvează asocierea în baza de date.

```
<form th:action="@{/asocieri}" th:object="${asociere}" method="post">

  <div class="mb-3">
    <label class="form-label">Alege Reteta</label>
    <select class="form-select" th:field="*{reteta}" required>
      <option value="" disabled selected>Selectează o rețetă</option>
      <option th:each="r : ${toateretetele}"
              th:value="${r.idReteta}"
              th:text="${r.numeReteta}"></option>
    </select>
  </div>

  <div class="mb-3">
    <label class="form-label">Alege Ingredient</label>
    <select class="form-select" th:field="*{ingredient}" required>
       <option value="" disabled selected>Selectează un ingredient</option>
       <option th:each="ing : ${toateingredientele}"
               th:value="${ing.idIngredient}"
               th:text="${ing.numeIngredient}"></option>
    </select>
  </div>

  <div class="row">
    <div class="col-md-6 mb-3">
      <label class="form-label">Cantitate</label>
      <input type="number" step="0.01" class="form-control" th:field="*{cantitate}" placeholder="ex: 200" required />
    </div>
    <div class="col-md-6 mb-3">
      <label class="form-label">Unitate</label>
      <input type="text" class="form-control" th:field="*{unitate}" placeholder="ex: grame, buc" required />
    </div>
  </div>

  <div class="d-grid gap-2 mt-4">
    <button type="submit" class="btn btn-primary btn-lg">Salvează</button>
  </div>

  <div class="d-flex justify-content-between mt-3">
    <a class="btn btn-outline-secondary" th:href="@{/asocieri}">Înapoi</a>
  </div>

</form>
```

*Fig. 18 – Formularul de adăugare compoziție rețete (**create_asociere.html**): utilizatorul selectează rețeta și ingredientul din liste, completează cantitatea și unitatea, iar la salvare datele sunt trimise către ruta **/asocieri** prin metoda POST*

```
@GetMapping("/asocieri/new")
public String createAsociereForm(Model model) {
    model.addAttribute("toateretete", retetaRepo.findAll());
    model.addAttribute("toateingredientele", ingredientRepo.findAll());
    model.addAttribute("asociere", new RetetaIngredient());
    return "create_asociere";
}

@PostMapping("/asocieri")
public String saveAsociere(@ModelAttribute("asociere") RetetaIngredient asociere) {
    retetaIngredientService.save(asociere);
    return "redirect:/asocieri";
}
```

*Fig. 19 – Implementarea adăugării în **RetetaIngredientController**: ruta GET **/asocieri/new** încarcă listele de rețete și ingrediente pentru select, iar ruta POST **/asocieri** preia datele formularului și salvează înscrierea în baza de date*

Pentru editarea unei înscrieri, utilizatorul apasă butonul „Modifică” din tabela de înscrieri, iar aplicația deschide formularul de modificare prin ruta GET **/asocieri/edit/{id}**, încărcând datele înscrierii selectate. După actualizarea rețetei, ingredientului, cantității sau unității, formularul trimite modificările către ruta POST **/asocieri/{id}**, iar controller-ul preia noile valori și actualizează înregistrarea în baza de date. La final, utilizatorul este redirecționat către pagina de listare a înscrierilor.

```
<form th:action="@{/asocieri/{id}(id=${asociere.idRetetaIngredient})}"
      th:object="${asociere}"
      method="post">

  <div class="mb-3">
    <label class="form-label">Reteta</label>
    <select class="form-select" th:field="*{reteta}" required>
      <option th:each="r : ${toateretete}"
              th:value="${r.idReteta}"
              th:text="${r.numeReteta}"></option>
    </select>
  </div>

  <div class="mb-3">
    <label class="form-label">Ingredient</label>
    <select class="form-select" th:field="*{ingredient}" required>
      <option th:each="ing : ${toateingredientele}"
              th:value="${ing.idIngredient}"
              th:text="${ing.numeIngredient}"></option>
    </select>
  </div>

  <div class="row">
    <div class="col-md-6 mb-3">
      <label class="form-label">Cantitate</label>
      <input type="number" step="0.01" class="form-control" th:field="*{cantitate}" required />
    </div>
    <div class="col-md-6 mb-3">
      <label class="form-label">Unitate</label>
      <input type="text" class="form-control" th:field="*{unitate}" required />
    </div>
  </div>

  <div class="d-grid gap-2 mt-4">
    <button type="submit" class="btn btn-primary btn-lg">Salvează modificările</button>
  </div>

  <div class="d-flex justify-content-between mt-3">
    <a class="btn btn-outline-secondary" th:href="@{/asocieri}">Renunță</a>
  </div>

</form>
```

*Fig. 20 – Formularul de editare înscriere (**edit_asociere.html**): la apăsarea butonului „Salvează modificările”, datele sunt trimise către ruta **/asocieri/{id}** prin metoda POST*

```java
@GetMapping("/asocieri/edit/{idasociere}")
public String editAsociereForm(@PathVariable Integer idasociere, Model model) {
    RetetaIngredient asociere = retetaIngredientService.findById(idasociere).orElse(null);
    if (asociere == null) return "redirect:/asocieri";

    model.addAttribute("asociere", asociere);
    model.addAttribute("toateretetele", retetaRepo.findAll());
    model.addAttribute("toateingredientele", ingredientRepo.findAll());

    return "edit_asociere";
}

@PostMapping("/asocieri/{idasociere}")
public String updateAsociere(@PathVariable Integer idasociere,
                             @ModelAttribute("asociere") RetetaIngredient asociere) {

    RetetaIngredient existent = retetaIngredientService.findById(idasociere).orElse(null);
    if (existent == null) return "redirect:/asocieri";

    existent.setReteta(asociere.getReteta());
    existent.setIngredient(asociere.getIngredient());
    existent.setCantitate(asociere.getCantitate());
    existent.setUnitate(asociere.getUnitate());

    retetaIngredientService.save(existent);
    return "redirect:/asocieri";
}
```

*Fig. 21 – Implementarea editării în **RetetaIngredientController**: ruta GET **/asocieri/edit/{id}** încarcă înscrierea și listele de rețete/ingrediente, iar ruta POST **/asocieri/{id}** preia modificările și actualizează înregistrarea în baza de date*

Pentru ștergerea unei înscrieri, utilizatorul apasă butonul „Șterge” din tabela de înscrieri. Înainte de executare, interfața poate solicita o confirmare pentru a evita ștergerea accidentală. Dacă utilizatorul confirmă, se apelează ruta **/asocieri/delete/{id}**, iar controller-ul elimină înregistrarea și revine la lista de înscrieri.

```html
<a class="btn btn-outline-danger btn-sm"
   th:href="@{/asocieri/delete/{id}(id=${asociere.idRetetaIngredient})}"
   onclick="return confirm('Sigur vrei să ștergi această compoziție?');">
  Șterge
</a>
```

*Fig. 22 – Butonul „Șterge” din **asocieri.html**: la confirmare, este apelată ruta **/asocieri/delete/{id}** pentru înscrierea selectată*

```java
@GetMapping("/asocieri/delete/{idasociere}")
public String deleteAsociere(@PathVariable Integer idasociere) {
    retetaIngredientService.deleteById(idasociere);
    return "redirect:/asocieri";
}
```

*Fig. 23 – Implementarea ștergerii în **RetetaIngredientController**: ruta GET **/asocieri/delete/{id}** apelează **deleteById(id)** și redirecționează către pagina de listare **(/asocieri)***

6. Interfața Grafică și Design (Front-End)

Deși accentul proiectului este pus pe logica de baze de date, interfața grafică joacă un rol crucial în uzabilitate. Pentru acest proiect, am implementat un design modern, bazat pe conceptul de **Glassmorphism**.

Elementele vizuale principale includ:

- **Fișier CSS centralizat (style.css):** Toate stilurile sunt definite într-un singur loc, asigurând consistența vizuală între pagini.
- **Fundal și Atmosferă:** Utilizarea unei imagini de fundal tematică (culinară), peste care s-a aplicat un filtru întunecat pentru a asigura lizibilitatea textului alb.
- **Carduri Translucide:** Containerele care găzduiesc tabelele și formularele au un fundal semi-transparent cu efect de blur (backdrop-filter: blur), oferind un aspect modern și "curat".
- **Feedback Vizual:** Butoanele își schimbă culoarea și poziția la interacțiunea cu mouse-ul (hover), iar butoanele de acțiune critică (Ștergere) sunt marcate distinct (roșu) față de cele de navigare sau salvare.

```css
/* 1. FUNDAL + LOGO */
body {
    min-height: 100vh;
    background: linear-gradient(180deg, rgba(0,0,0,0.5), rgba(0,0,0,0.8)),
                url("images/Retete.png") center/cover no-repeat fixed;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    color: white;
    display: flex;
    flex-direction: column;
    align-items: center;
    padding-top: 20px;
}

/* LOGO */
body::before {
    content: "";
    display: block;
    position: fixed;
    top: 20px;
    left: 20px;
    width: 90px;
    height: 90px;
    background-image: url('images/Sigla_Facultate_ETTI.webp');
    background-size: contain;
    background-repeat: no-repeat;
    background-position: center;
    z-index: 9999;
    filter: drop-shadow(2px 4px 6px rgba(0,0,0,0.5));
}

/* 2. TEXT */
body, h1, h2, h3, h4, h5, h6, p, span, div, a, th, td, label, button {
    text-shadow: 1px 1px 2px rgba(0, 0, 0, 0.9);
    letter-spacing: 0.3px;
}

/* 3. DESIGN "GLASSMORPHISM" */
.glass, .card {
    background: rgba(40, 40, 40, 0.6) !important;
    border: 1px solid rgba(255, 255, 255, 0.15) !important;
    border-radius: 18px !important;
    backdrop-filter: blur(15px);
    -webkit-backdrop-filter: blur(15px);
    box-shadow: 0 15px 35px rgba(0, 0, 0, 0.5) !important;
    color: white !important;
}

.card-body {
    background: transparent !important;
}
```

```css
/* 4. BUTOANE */
.btn {
    border: 1px solid rgba(255,255,255,0.4) !important;
    background-color: rgba(255,255,255,0.05) !important;
    color: white !important;
    border-radius: 8px;
    font-weight: 600;
    font-size: 0.85rem;
    transition: all 0.2s ease-in-out;
    text-transform: uppercase;
    padding: 6px 12px;
}

.btn:hover {
    transform: translateY(-2px);
    background-color: rgba(255, 255, 255, 0.2) !important;
    border-color: #fff !important;
    color: #fff !important;
    box-shadow: 0 5px 15px rgba(0,0,0,0.3);
}

.btn-outline-danger:hover {
    background-color: rgba(220, 53, 69, 0.6) !important;
    border-color: #ff8fa3 !important;
}

/* --- 5. TABELE --- */

.table-responsive {
    background: rgba(0, 0, 0, 0.4);
    backdrop-filter: blur(20px);
    -webkit-backdrop-filter: blur(20px);

    border-radius: 15px;
    border: 1px solid rgba(255, 255, 255, 0.1);
    box-shadow: 0 8px 32px rgba(0, 0, 0, 0.4);

    overflow: hidden;
}

.table {
    --bs-table-bg: transparent !important;
    --bs-table-color: white !important;
    border-color: rgba(255, 255, 255, 0.1) !important;
    margin-bottom: 0;
    color: white !important;
}
```

```
/* HEADER TABEL */
.table thead th {
    background-color: rgba(0, 0, 0, 0.45) !important;
    color: #ffe082 !important;
    text-transform: uppercase;
    font-size: 0.8rem;
    padding: 15px 10px;
    border-bottom: 1px solid rgba(255, 224, 130, 0.2);

    /* Centrare HEADER */
    text-align: center !important;
    vertical-align: middle !important;
    white-space: nowrap;
}

/* CELULE TABEL */
.table tbody td {
    color: white !important;
    vertical-align: middle !important;
    padding: 10px 10px;
    border-bottom: 1px solid rgba(255,255,255,0.08);
    background: transparent !important;

    /* CENTRARE CONTINUT CELULE */
    text-align: center !important;
}

/* Coloana Actiuni */
.table td:last-child {
    white-space: nowrap !important;
    width: 1%;
}

/* Centrare grup butoane actiuni */
.table td:last-child .d-flex {
    justify-content: center !important;
    flex-wrap: nowrap !important;
}
```

```
/* Randuri impare */
.table-striped > tbody > tr:nth-of-type(odd) > * {
    background-color: rgba(255, 255, 255, 0.03) !important;
    color: white !important;
    --bs-table-accent-bg: transparent !important;
}

/* Hover pe rand */
.table-striped > tbody > tr:hover > * {
    background-color: rgba(255, 224, 130, 0.1) !important;
    color: white !important;
}

/* 6. INPUTURI FORMULARE */
.form-control, .form-select, textarea {
    background-color: rgba(0, 0, 0, 0.6) !important;
    border: 1px solid rgba(255, 255, 255, 0.3) !important;
    color: white !important;
    border-radius: 8px;
}

.form-control:focus, .form-select:focus {
    background-color: rgba(0, 0, 0, 0.8) !important;
    border-color: #ffe082 !important;
    box-shadow: 0 0 10px rgba(255, 224, 130, 0.2);
    color: white !important;
}

::placeholder {
    color: rgba(255, 255, 255, 0.6) !important;
}
```

Fig. 24:27 – Fișier style.css

7. Concluzii

Acest proiect a demonstrat, în mod practic, eficiența utilizării ecosistemului Java Spring Boot pentru dezvoltarea unei aplicații web conectate la o bază de date relațională MySQL. Implementarea unei „Cărți de Bucate Digitale” evidențiază integrarea coerentă dintre stratul de persistență, logica de business și interfața de prezentare, într-o arhitectură de tip MVC (Model–View–Controller).

Din perspectiva modelării datelor, soluția ilustrează utilizarea corectă a unei relații Many-to-Many (M:N) între entitățile Rețete și Ingrediente, realizată printr-o tabelă de legătură ce stochează și atribute suplimentare, precum cantitatea și unitatea. Această structură reflectă un scenariu realist și susține integritatea referențială prin chei străine și constrângeri specifice.

La nivel funcțional, aplicația permite operații CRUD complete pentru tabelele principale și pentru entitatea de asociere. Accesul la date este realizat prin Spring Data JPA/Hibernate, ceea ce simplifică implementarea și reduce codul repetitiv, iar interfața construită cu Thymeleaf asigură un flux clar de lucru prin pagini de listare și formulare dedicate.

Aplicația poate fi extinsă în mod natural prin adăugarea unui modul de autentificare și autorizare, precum și prin validări suplimentare la nivel de backend (de exemplu, verificarea cantităților pozitive și standardizarea unităților). De asemenea, funcții precum căutarea și filtrarea după alergeni, paginarea și sortarea ar îmbunătăți uzabilitatea și robustețea aplicației.

În concluzie, proiectul își atinge obiectivele stabilite, oferind o aplicație web coerentă și extensibilă, care demonstrează aplicarea corectă a conceptelor de programare a interfețelor către baze de date, atât la nivel de modelare relațională, cât și la nivel de implementare în Spring Boot.

8. Bibliografie

1. Valentin Pupezescu - **Curs_Java Spring Boot**, suport de curs BDPI (SpringToolsForEclipse, Tomcat, MySQL, CRUD).

2. Walls, Craig. *Spring Boot in Action*. Manning Publications, 2016.

3. Johnson, Rod et al. *Spring Framework Documentation*. [Online]. Disponibil la: https://docs.spring.io/spring-framework/docs/current/reference/html/

4. Oracle Corporation. *MySQL 8.0 Reference Manual*. [Online]. Disponibil la: https://dev.mysql.com/doc/refman/8.0/en/

5. Thymeleaf Team. *Thymeleaf Documentation - Tutorial: Using Thymeleaf*. [Online]. Disponibil la: https://www.thymeleaf.org/doc/tutorials/3.0/usingthymeleaf.html

6. MDN Web Docs. *CSS: Cascading Style Sheets*. [Online]. Disponibil la: https://developer.mozilla.org/en-US/docs/Web/CSS